Universidad Corporativa Coppel | Programas Académicos

Ingeniería en Desarrollo de Software

Actividad | 3 | código en lenguaje C

Desarrollo de software

Ingeniería en Desarrollo de Sofwtare

academiaglobal

TUTOR: Sandra Luz Lara Dévora

ALUMNO: Jesus Christopher Enríquez Sotelo

FECHA: 20/03/2026

Índice

1 Portada

2 Índice

3 Introducción

4 Descripción de la actividad

5 Justificación

5 Desarrollo
6.1. Algoritmo de números primos
6.2. Lógica del algoritmo de números primos
6.3. Algoritmo de números pares e impares
6.4. Lógica del algoritmo de números pares e impares
6.5. Algoritmo de números invertidos
6.6. Lógica del algoritmo de números invertidos

Introducción:

En esta actividad se trabaja el desarrollo de algoritmos como una herramienta fundamental para resolver problemas de forma ordenada y lógica. El objetivo principal es comprender cómo se estructura un proceso antes de convertirlo en un programa más formal. A partir de tres ejercicios básicos, se analiza la manera en que una computadora puede identificar si un número es primo, distinguir entre números pares e impares y obtener un número invertido. Este tipo de ejercicios permite practicar el pensamiento lógico y la organización de pasos, que son esenciales en programación. Además, ayudan a fortalecer la capacidad de interpretar problemas matemáticos y traducirlos a instrucciones claras. La actividad también sirve como base para futuras etapas, ya que primero se entiende el algoritmo y después se puede representar en diagrama de flujo o en código. Por ello, su importancia no solo está en resolver los ejercicios, sino en aprender a pensar como programador.

Descripción de la actividad:

La actividad consiste en elaborar tres algoritmos diferentes, cada uno enfocado en una función específica. El primero debe analizar si un número es primo, es decir, si solo puede dividirse entre 1 y entre sí mismo. El segundo debe recibir varios números y determinar cuáles son pares y cuáles son impares, usando la operación módulo para verificar el residuo de la división entre dos. El tercero debe tomar un número de cuatro dígitos y mostrarlo invertido, procesando sus cifras una por una. Para cada uno de estos casos, no solo se debe escribir el algoritmo, sino también explicar la lógica utilizada en cada línea, para demostrar que se entiende el funcionamiento del procedimiento. La actividad está pensada para que el estudiante practique la forma en que se construyen soluciones paso a paso, con instrucciones ordenadas y comprensibles. También permite relacionar la teoría con la práctica, ya que cada algoritmo representa una forma distinta de resolver una situación concreta.

Justificación:

Esta actividad es importante porque permite desarrollar habilidades básicas de programación y razonamiento lógico. Aprender a crear algoritmos ayuda a dividir un problema grande en partes pequeñas y manejables, lo cual facilita encontrar una solución clara y eficiente. En este caso, los tres ejercicios propuestos son útiles porque trabajan conceptos diferentes: validación de divisores, uso de ciclos, operaciones con módulo y manipulación de dígitos. Estas prácticas fortalecen la

comprensión de estructuras fundamentales que después se usan en lenguajes de programación más avanzados. También es una actividad valiosa porque obliga a explicar la lógica de cada paso, no solo a escribir código o pseudocódigo. Eso ayuda a entender realmente qué hace cada instrucción y por qué se usa. Además, este tipo de ejercicios prepara al estudiante para resolver problemas más complejos en el futuro, ya que enseña a pensar de manera estructurada y ordenada. Por eso, la actividad no solo cumple una función académica, sino formativa, porque construye bases sólidas para seguir aprendiendo programación.

Desarrollo:

En el desarrollo de esta actividad se presentan tres algoritmos que resuelven problemas matemáticos básicos. Cada uno incluye su procedimiento y la explicación de la lógica utilizada, con el fin de comprender cómo se estructura una solución paso a paso.

Programa números primos:

```c
#include <stdio.h>

int main() {
    int numeroingresado, iteracion = 1, divisionresiduocera = 0;

    printf("Ingresa un numero: ");
    scanf("%d", &numeroingresado);

    while(iteracion <= numeroingresado) {
        if(numeroingresado % iteracion == 0) {
            divisionresiduocera = divisionresiduocera + 1;
        }
        iteracion = iteracion + 1;
    }

    if(divisionresiduocera == 2) {
        printf("El numero es primo");
    } else {
        printf("El numero no es primo");
    }

    return 0;
}
```

Explicación línea por línea

#include <stdio.h>
Permite usar funciones como printf y scanf.

int main() {
Es la función principal donde inicia el programa.

int numeroingresado, iteracion = 1, divisionresiduocera = 0;
Se declaran variables:

- **numeroingresado: guarda el número**
- **iteracion: recorre los divisores**
- **divisionresiduocera: cuenta divisiones exactas**

printf("Ingresa un numero: ");
Muestra un mensaje al usuario.

scanf("%d", &numeroingresado);
Lee el número ingresado.

while(iteracion <= numeroingresado)
Recorre desde 1 hasta el número ingresado.

if(numeroingresado % iteracion == 0)
Verifica si es divisible sin residuo.

divisionresiduocera = divisionresiduocera + 1;
Cuenta los divisores.

iteracion = iteracion + 1;
Avanza al siguiente número.

if(divisionresiduocera == 2)
Si tiene 2 divisores, es primo.

printf("El numero es primo");
Muestra resultado.

else
Caso contrario.

printf("El numero no es primo");
Muestra que no es primp

```
Ingresa un numero: 12
El numero no es primo

...Program finished with exit code 0
Press ENTER to exit console.
```

Programa numero par e impar:

```c
#include <stdio.h>

int main() {
    int num = 0, nro;

    while(num < 10) {
        num = num + 1;

        printf("Ingresa un numero: ");
        scanf("%d", &nro);

        if(nro % 2 == 0) {
            printf("Es par\n");
        } else {
            printf("Es impar\n");
        }
    }

    return 0;
}
```

Explicación línea por línea

int num = 0, nro;
num cuenta repeticiones, nro guarda el número.

while(num < 10)
Repite el proceso 10 veces.

num = num + 1;
Aumenta el contador.

scanf("%d", &nro);
Lee el número.

if(nro % 2 == 0)
Verifica si es divisible entre 2.

printf("Es par");
Muestra que es par.

else
Caso contrario.

printf("Es impar");
Muestra que es impar.

```
Ingresa un numero: 5
Es impar
Ingresa un numero: 6
Es par
Ingresa un numero: 7
Es impar
Ingresa un numero: 8
Es par
Ingresa un numero: 9
Es impar
Ingresa un numero: 10
Es par


...Program finished with exit code 0
Press ENTER to exit console.
```

Programa números invertidos:

```c
#include <stdio.h>

int main() {
    int numero, inverso = 0, residuo;

    printf("Ingresa un numero: ");
    scanf("%d", &numero);

    residuo = numero;

    while(residuo > 0) {
        inverso = inverso * 10 + residuo % 10;
        residuo = (residuo - residuo % 10) / 10;
    }

    printf("Valor del numero inverso: %d\n", inverso);
    printf("El numero original es: %d", numero);

    return 0;
}
```

Explicación línea por línea

int numero, inverso = 0, residuo;
Variables para número, inverso y auxiliar.

scanf("%d", &numero);
Lee el número.

residuo = numero;
Copia el número original.

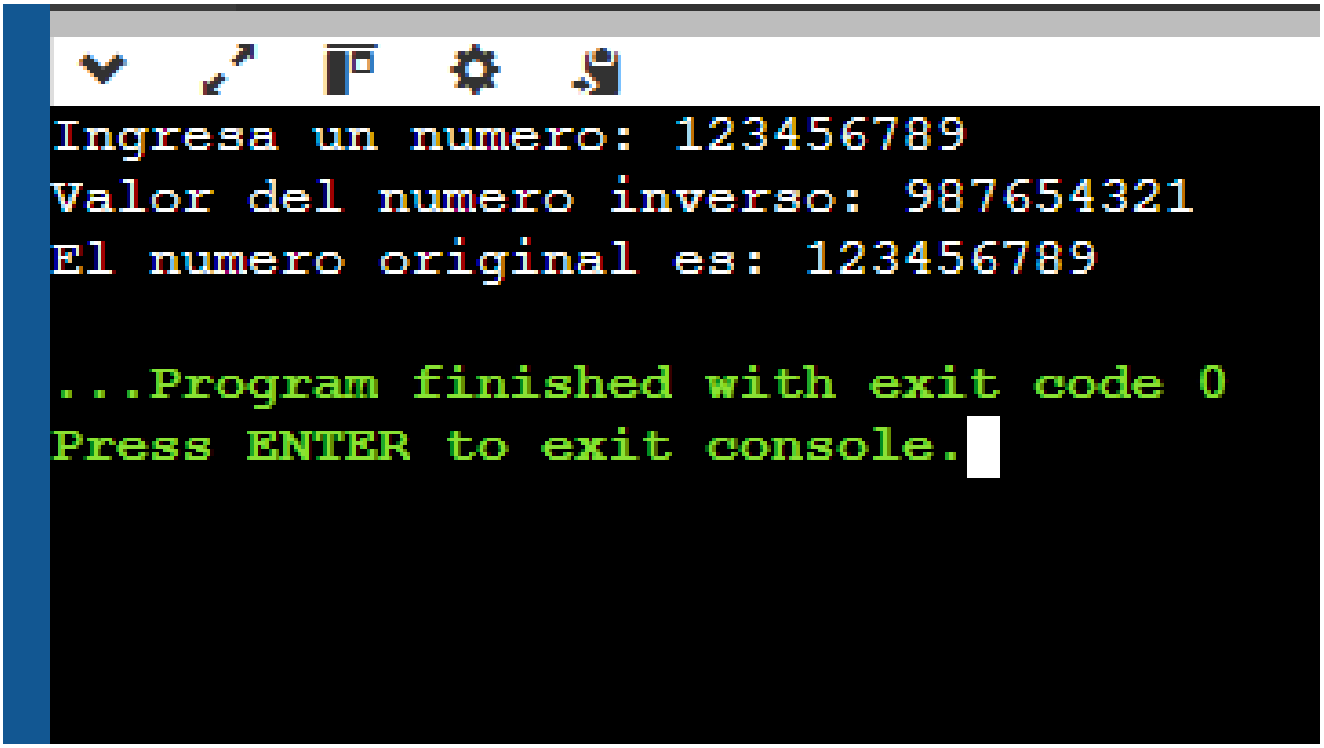
while(residuo > 0)
Mientras haya dígitos.

inverso = inverso * 10 + residuo % 10;
Agrega el último dígito al inverso.

residuo = (residuo - residuo % 10) / 10;
Elimina el último dígito.

printf("Valor del numero inverso...");
Muestra el número invertido.

printf("El numero original...");
Muestra el número original.



Conclusión

En conclusión, la realización de esta actividad permitió comprender de manera más profunda la importancia de los algoritmos en la resolución de problemas. A través del desarrollo de los tres ejercicios, se pudo observar cómo un problema puede descomponerse en pasos claros y ordenados, facilitando su solución. Los algoritmos de números primos, pares e impares, y números invertidos ayudaron a reforzar conceptos clave como el uso de ciclos, condiciones y operaciones matemáticas como el módulo. Además, el hecho de explicar cada línea permitió entender realmente el funcionamiento del proceso, y no solo escribir instrucciones sin sentido. Esta práctica fortalece el pensamiento lógico y la capacidad de análisis, habilidades fundamentales en la programación. Asimismo, el uso de diagramas de flujo complementa el aprendizaje al ofrecer una representación visual del proceso. En general, esta actividad contribuye a desarrollar bases sólidas que serán útiles en futuros proyectos y en el aprendizaje de lenguajes de programación más avanzados.

Link git hub: https://github.com/kessoChris/desarrollo-de-software-